

个人作业报告

题目：**大数据原理与技术**

作业七：混合推荐系统设计

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	王昌栋

2026 年 4 月 30 日

【摘　要】

在学习王老师的大数据原理与技术课程[1]后，在此报告中完成相应的课后作业。本作业围绕混合推荐系统设计作业展开。

任务要求：

- 基于 MovieLens 数据集
- 实现协同过滤（用户/物品相似度）与基于内容的推荐（电影标签），并比较两者的召回率；
- 设计加权混合策略或其他策略以提升推荐多样性。

提交要求：

- 提交内容：代码 + 简单报告（至少 2 页）
- 命名格式：学号 + 姓名 + 作业 7.zip
- 提交至邮箱：BDSysu2026Spring@163.com
- 截止时间：布置作业之后的下一个周日 24:00
- 严禁抄袭：任何一次抄袭，平时成绩按零分处理

本报告基于 MovieLens（ml-latest-small）数据集，设计并实现了一个可解释的混合电影推荐系统。系统包括：1) 基于用户/物品相似度的协同过滤（User-CF 与 Item-CF）；2) 基于电影类型（genres）的内容推荐（TF-IDF + 余弦相似度）；3) 将上述子系统以加权混合策略融合，并在冷启动场景引入流行度兜底与多样性惩罚机制。

报告中详细记录了数据预处理、推荐系统设计、候选召回与融合流程、实验结果与案例分析等内容。实验结果表明，混合策略在多数示例用户上能提升 Recall@20，且在历史记录稀少的用户上通过热度补偿提高覆盖率；本次作业实现了“召回 + 融合 + 重排”流水线的工程，也进行了完备的实验效果分析。

关键词： 混合推荐系统设计，协同过滤，基于内容的推荐，召回率，多样性提升

目录

1 混合电影推荐系统实验 1
1.1 数据集与预处理 1
1.2 推荐系统模型设计 1
1.3 算法详解 2
1.4 实验结果 6
1.5 结果分析 9

2 总结 10
2.1 实验总结 10
2.2 心得体会 10

参考文献 11

附录 A 附录: 实现代码 12

1 混合电影推荐系统实验

1.1 数据集与预处理

1.1.1 数据来源

使用 MovieLens 数据集（工程中为 `ml-latest-small` 子集），脚本加载输出为：9742 部电影，100836 条评分。

1.1.2 预处理流程

主要步骤：

1) 用评分表构造用户-物品矩阵（pivot，缺失填 0）；
2) 计算用户相似度：对用户-物品稠密矩阵使用余弦相似度；
3) 计算物品相似度：对物品-用户矩阵使用余弦相似度；
4) 内容相似度：对电影的 `genres` 字段使用 TF-IDF，计算余弦相似度矩阵；
5) SVD 降维（TruncatedSVD，n_components=50）用于降维/加速（当前实现为演示用途）。

1.2 推荐系统模型设计

总体设计思想以可解释、易实现的混合推荐为主线，先用协同过滤捕捉用户群体偏好，再用基于内容的相似度补充长尾与冷启动场景。为兼顾计算效率与可复现性，离线预计算用户/物品相似度与内容相似度矩阵，在线阶段按“候选召回 + 加权融合 + 多样性重排”的流水线生成推荐；当用户历史稀少时，引入热门度作为兜底信号提升覆盖率。

实现要点：

- **用户协同过滤 (user-based CF)**：用用户相似度选择 top-N 相似用户，按相似度和对方评分汇总候选电影；
- **物品协同过滤 (item-based CF)**：对用户已评分电影查找相似电影并累加评分贡献；
- **内容推荐**：对用户最近观看的若干电影使用基于类型的相似度返回候选；
- **混合策略**：按权重合并得分，默认权重为 $\text{cf_weight} = 0.6, \text{content_weight} =$

0.4；冷启动用户（历史少于 5 条）会引入热度权重 (popularity)。

1.3 算法详解

下面对重点算法与实现细节做深入说明，便于理解当前系统的设计与后续改进方向。

1.3.1 协同过滤（Collaborative Filtering）

协同过滤通过用户历史评分的相似性来实现个性化推荐，常见形式有基于用户（user-based）和基于物品（item-based）两类。

1.3.1.1 基于用户的协同过滤

设 $r_{u,i}$ 为用户 u 对物品 i 的评分，$\bar{r}_u$ 为用户 u 的平均评分，$N(u)$ 为与 u 相似的近邻用户集合，则基于用户的预测可以写为：

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(u)} s(u,v)\,(r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} |s(u,v)|},$$

其中 $s(u,v)$ 为用户相似度（见下节）。该公式能校正不同用户评分尺度的偏差（通过去均值处理）。实现中也常采用不去均值的简化形式：直接按相似度加权对相邻用户的评分求和并归一化。

本实现采用不去均值的简化形式，并省略归一化项，只对 Top-k 相似用户累加相似度加权评分：

$$\text{score}_{u,i} = \sum_{v \in N_k(u)} s(u,v)\, r_{v,i}.$$

其中 $k = 10$，仅对用户未评分的电影进行排序推荐。

1.3.1.2 基于物品的协同过滤

物品协同过滤把每个物品视为一个向量（在用户维度上），对用户已评分物品查找最相似的物品并累加贡献：

$$\hat{r}_{u,i} = \frac{\sum_{j \in S(u)} s(i,j)\, r_{u,j}}{\sum_{j \in S(u)} |s(i,j)|},$$

其中 $S(u)$ 为用户 u 已评分的物品集合，$s(i,j)$ 为物品相似度。物品相似度通常可预先计算并缓存，适合在线推荐场景。

当前实现同样采用简化的加权求和，不做归一化：

$$\text{score}_{u,i} = \sum_{j \in R(u)} s(i,j)\, r_{u,j}.$$

实际代码对用户已评分电影逐个取 Top-k 相似物品（$k = 10$）并累加得分，最后按得分排序推荐。

1.3.2 相似度度量与改进

常见相似度包括：

- 余弦相似度（Cosine）：当将评分向量 $\mathbf{r}_u$ 视为实向量时，

$$s_{cos}(u,v) = \frac{\mathbf{r}_u \cdot \mathbf{r}_v}{\|\mathbf{r}_u\|\|\mathbf{r}_v\|}.$$

  在稀疏评分中通常只对共同评分的项（co-rated items）计算余弦值，以避免零填充的偏差。

- 皮尔逊相关（Pearson）：对用户评分做中心化处理后衡量线性相关性，公式为：

$$s_{pearson}(u,v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2}},$$

  其中 I_{uv} 为 u, v 的共同评分集合。

- 收缩/正则化（Shrinkage）：当共同评分数 n_{uv} 很小时，相似度的不确定性较大，可用收缩策略降低权重：

$$s_{shrunk} = \frac{n_{uv}}{n_{uv} + \lambda}\, s(u,v),$$

  其中 λ 控制收缩强度。

在实现中，选择相似度时要注意是否需要去均值、是否仅对共同评分计算，以及如何对相似度做阈值截断或 Top-K 保留以减少计算与噪声影响。

本代码中用户相似度与物品相似度直接对缺失填 0 的稠密矩阵计算余弦相似度，因此未显式只取共同评分项，也未加入收缩修正；实际采用 Top-k 截断来控制噪声与计算量。

1.3.3 矩阵分解（Matrix Factorization）

显式评分预测常用矩阵分解方法，将评分矩阵 R 分解为低秩因子：

$$R \approx PQ^T, \quad P \in \mathbb{R}^{|U|\times f},\ Q \in \mathbb{R}^{|I|\times f}.$$

预测为 $\hat{r}_{u,i} = p_u^T q_i$，训练通过最小化有正则项的平方损失：

$$L = \sum_{(u,i)\in K} (r_{u,i} - p_u^T q_i)^2 + \lambda\big(\|p_u\|^2 + \|q_i\|^2\big).$$

可用随机梯度下降（SGD）或交替最小二乘（ALS）优化。矩阵分解在稀疏数据上的表现通常优于基于邻域的简单 CF，且能自然刻画隐语义特征。

当前实现中仅计算了 TruncatedSVD 作为降维与加速的演示，未将分解结果用于评分预测或重排。

1.3.4 基于内容的推荐（Content-based）

基于内容的方法把物品表示为特征向量（如从类型、简介、标签、演员等构建），对电影类型使用 TF-IDF 向量化并计算余弦相似度：

$$s_{content}(i,j) = \frac{v_i \cdot v_j}{\|v_i\|\|v_j\|}.$$

此外，可为用户构建兴趣向量（对用户喜欢的电影特征向量加权平均），用 $\text{score}(i|u) = \text{sim}(v_i, v_u)$ 排序推荐。

本实现仅使用 `genres` 字段构建 TF-IDF，并预计算电影间相似度矩阵。对用户选取最近 3 部已评分电影，各自取 Top-K 相似电影形成候选，并以出现次数作为内容得分：

$$\text{score}_{content}(i|u) = \sum_{m\in R_{recent}(u)} \mathbb{I}[i \in \text{TopK}(m)].$$

1.3.5 混合策略详述

常见混合方法包括：

- 加权混合（Weighted）：对各个子系统输出分数归一化后按权重线性组合：

$$S(i|u) = \alpha\, S_{cf}(i|u) + \beta\, S_{content}(i|u) + \gamma\, S_{pop}(i), \quad \alpha+\beta+\gamma = 1.$$

- 切换式（Switching）：根据用户历史量级或置信度选择使用 CF 或内容推荐；
- 级联/候选-重排（Cascade / Candidate-ReRank）：先用高召回的轻量模型（如流行度或 Item-CF）召回候选，再用更精细的排序模型（学习到的回归/排序器）进行重排；
- 特征组合（Feature-Combination）与学习融合（Learning-to-Fuse）：将 CF 得分、内容特征以及用户/物品元特征拼接，训练一个监督排序模型（如 LambdaMART）学习最佳组合权重。

本实现采用加权混合，将各子系统的候选出现次数作为分数：

$$S(i|u) = w_{cf}\, c_{cf}(i|u) + w_{content}\, c_{content}(i|u) + w_{pop}\, c_{pop}(i).$$

其中 c_{cf} 来自用户 CF 与物品 CF 的候选合并计数，$c_{content}$ 来自最近 3 部电影的内容候选计数，c_{pop} 表示是否属于全局评分数 Top-K。默认 $w_{cf} = 0.6$, $w_{content} = 0.4$, $w_{pop} = 0$；当用户历史评分少于 5 条时，权重调整为 0.4/0.3/0.3 以引入热度补偿。

实现注意：当前融合未做显式分数归一化，而是以候选计数作为统一尺度，便于快速混合。

1.3.6 多样性与再排序（Diversity & Re-ranking）

为了避免推荐列表过于集中在同一类型，可采用最大边际相关性（MMR）等增量选择策略：

$$\text{MMR} = \arg\max_{i \in C \setminus S} \Big[\lambda\, \text{rel}(i) - (1-\lambda) \max_{j \in S} \text{sim}(i,j) \Big],$$

其中 C 为候选集，S 为已选集合，$\text{rel}(i)$ 为相关性分数，$\text{sim}(i,j)$ 为物品相似度，λ 控制多样性与相关性的权衡。

当前实现采用简化的类型多样性惩罚：

$$\text{score}'(i) = \text{score}(i) - 0.1 \sum_{g \in G(i)} \text{count}(g),$$

其中 $G(i)$ 为电影 i 的类型集合，$\text{count}(g)$ 为已选列表中类型 g 出现次数，仅保留 $\text{score}'(i) > 0$ 的候选。

1.3.7 冷启动、稀疏性与可扩展性

- 冷启动：本实现中当用户历史少于 5 条时引入热度推荐，热度由电影评分次数的全局 Top-K 提供；对新物品未做专门冷启动；
- 稀疏性：矩阵分解、默认评分推断（imputation）和组合特征都能缓解稀疏性；
- 可扩展性：使用稀疏矩阵、局部近邻（Top-K）预计算、近似最近邻检索（Faiss/Annoy）、并行/分布式训练提升大规模可用性。

1.3.8 离线评估协议

1) 对单个用户，取其最近的 $\lfloor n \cdot test_ratio \rfloor$ 条评分作为测试集；
2) 为避免数据泄露，评估时会临时从全局评分表中剔除这些测试评分，重建训练时用的用户-物品矩阵与相似度矩阵（即在训练数据上生成推荐），随后恢复原始评分；
3) 推荐列表长度在评估中使用 $K = 20$（函数中默认），命中数定义为 |recommendations∩ test_movies|，命中率为命中数除以测试集大小；

1.4 实验结果

1.4.1 召回率比较

在对样例用户 {4,16,27,69,99,101,110,120,134,180} 的离线评估中，我们比较了混合推荐（hybrid）与三种子系统的召回率（Recall@20）。下表列出每位用户的测试集大小以及混合推荐与各子系统的召回率：

用户 ID	测试集大小	混合	用户 CF	物品 CF	内容
4	43	2.33%	2.33%	2.33%	2.33%
16	19	15.79%	10.53%	10.53%	0.00%
27	27	7.41%	7.41%	11.11%	3.70%
69	9	33.33%	22.22%	11.11%	0.00%
99	10	20.00%	20.00%	20.00%	10.00%
101	12	25.00%	8.33%	8.33%	0.00%
110	10	20.00%	20.00%	0.00%	10.00%
120	4	25.00%	0.00%	0.00%	0.00%
134	7	14.29%	0.00%	14.29%	14.29%
180	4	75.00%	50.00%	50.00%	25.00%

表 1.1 样例用户的 Recall@20 比较（混合 vs 子系统）

总体观察分析：

- 对多数用户而言，用户/物品协同过滤在召回上通常优于纯内容推荐；混合策

略在大多数样例中能保持或提升召回，尤其在用户历史较丰富时效果明显。

- 内容推荐在样本稀疏或用户偏好更依赖类型相似时表现较好，但总体召回不稳定。
- 个别用户（如 120）出现子系统召回为 0 而混合推荐仍有命中，说明混合中引入的流行度/补偿策略在冷启动场景能提高覆盖。

1.4.2 结果命中率

1.4.2.1 优化前

下面展示对若干示例用户的离线评估结果（脚本运行日志摘录，使用样例用户集 {4,16,27,69,99,101,110,120,134,180}）：

用户 ID	历史评分数	测试集大小	命中数	Recall@20
4	216	43	1	2.33%
16	98	19	2	10.53%
27	135	27	2	7.41%
69	46	9	0	0.00%
99	53	10	2	20.00%
101	61	12	2	16.67%
110	51	20	1	5.00%
120	22	4	0	0.00%
134	35	7	0	0.00%
180	24	4	1	25.00%

表 1.2 示例用户的离线评估结果（Recall@20）

1.4.2.2 优化后

采用加权混合策略后，重新评估同一批示例用户的离线结果如下：

用户 ID	历史评分数	测试集大小	命中数	Recall@20
4	216	43	1	2.33%
16	98	19	3	15.79%
27	135	27	2	7.41%
69	46	9	3	33.33%
99	53	10	2	20.00%
101	61	12	3	25.00%
110	51	20	2	20.00%
120	22	4	1	25.00%
134	35	7	1	14.29%
180	24	4	3	75.00%

表 1.3 示例用户的离线评估结果（Recall@20）

结果分析　从对比可见，优化后 10 名用户中有 7 名 Recall@20 提升，3 名保持不变，未出现下降。提升最明显的用户包括 69、110 与 180，说明加权混合策略对原本召回较低的用户具有显著补偿作用；而用户 4、27、99 的结果基本持平，提示在偏好稳定或历史较充分的用户上增益有限。需要注意的是，部分用户测试集规模较小（如 4、120、180），Recall 波动较大，结论仍需通过更大规模用户采样或多次切分进行稳健性验证。

1.4.3　典型案例分析

选取 3 个代表性用户进行深入分析，案例来自运行日志的混合推荐结果与离线评估指标：

1.4.3.1　用户 16：中等历史、协同过滤主导

- 评分概况：历史评分 98 条，最近观看为 Full Metal Jacket (4.5)、Good Will Hunting (3.5)、Forrest Gump (3.5)。
- 推荐摘要：Godfather: Part II、The Departed、Saving Private Ryan、Braveheart、Star Wars: Episode VI 等，偏向战争/犯罪/史诗类。
- 离线评估：测试集大小 19，命中 3，Recall@20 为 15.79%，Precision 为 17.65%；User-CF 与 Item-CF Recall 均为 10.5%。
- 结论：内容侧基于 genres 的信号较弱，混合结果主要由协同过滤贡献。

1.4.3.2　用户 69：高评分偏好、混合召回提升

- 评分概况：历史评分 46 条，最近观看为 Cast Away (4.0)、O Brother, Where Art Thou? (5.0)、The Sixth Sense (5.0)。
- 推荐摘要：Raiders of the Lost Ark、Braveheart、Gladiator、Back to the Future、The Godfather 等，动作/冒险/剧情为主。
- 离线评估：测试集大小 9，命中 3，Recall@20 为 33.33%，Precision 为 15.00%；User-CF Recall 22.2%，Item-CF Recall 11.1%。
- 结论：协同过滤能较好捕捉该用户的主流类型偏好，混合策略进一步提高召回。

1.4.3.3 用户 120：冷启动场景、热度兜底

- 评分概况：历史评分 22 条，最近观看为 Jack (3.0)、The Godfather (5.0)、101 Dalmatians (4.0)。
- 推荐摘要：Toy Story、Eraser、The Nutty Professor、Mr. Holland's Opus、Multiplicity 等，偏向 90 年代热门片。
- 离线评估：测试集大小 4，命中 1，Recall@20 为 25.00%，Precision 为 5.00%；User-CF、Item-CF、内容推荐 Recall 均为 0。
- 结论：在历史较少的情况下，混合策略通过热度补偿提供有效覆盖，但个性化程度仍有限。

1.5 结果分析

- 实验结果表明，当前推荐系统存在显著的流行度偏差（Popularity Bias），热门电影在推荐列表中出现频率过高，这可能影响长尾物品的曝光机会和推荐多样性；
- 协同过滤部分在当前实现中未充分利用评分幅度（仅以出现计数），导致对用户特定偏好的区分度不足；
- 对于评分样本稀疏或冷启动用户，内容推荐与流行度补偿能提高覆盖但牺牲个性化；
- 评价指标单一（仅 Recall@K）可能无法全面反映排序质量，建议增加 Precision@K、NDCG@K、MAP 等并考虑按高评分（例如 rating>=4）定义正例。

2 总结

2.1 实验总结

本实验基于 MovieLens 小规模数据集实现了一个可解释的混合电影推荐系统，整体流程包含数据预处理、相似度计算、候选召回、加权融合与多样性重排。系统融合了用户 CF、物品 CF 与基于类型的内容推荐，并在冷启动场景中引入热门度兜底，提高覆盖率。离线评估采用按时间切分的单用户测试集，评估时剔除测试评分以避免数据泄露；在样例用户上混合策略总体优于单一方法，召回率得到提升，但仍存在流行度偏差和个体用户波动。

整体来看，当前实现能够较好地展示推荐系统的核心机制与工程结构：协同过滤提供群体偏好信号，内容相似度补充长尾与稀疏情形，多样性惩罚避免列表同质化。与此同时，分数未归一化、评分幅度利用不足和评估样本较小仍限制了效果与结论的稳健性。

2.2 心得体会

通过本次实现，我对“召回 + 融合 + 重排”的推荐流水线有了更清晰的认识：即便是简单的 Top-k 相似度与计数式融合，也能构成一个可用的推荐框架。实践中，评估设置（如是否剔除测试评分）对结果影响很大，修正评估流程后召回率明显改善，说明严格的离线评估是可靠结论的前提。

后续可以从以下方向继续完善：引入去均值或归一化以提升 CF 稳定性；在内容侧引入更多字段（标签、简介）增强个性化；增加 Precision@K、NDCG@K 等指标并扩大用户采样范围；在工程上使用稀疏矩阵或近似近邻加速相似度计算。

整体而言，本次实验验证了混合推荐的有效性，也暴露了在数据稀疏与分数标定方面的改进空间。

参考文献

[1] 王昌栋. 第一章绪论[Z]. 2026.

附录 A 附录：实现代码

```
"""
Hybrid Movie Recommendation System
Combines collaborative filtering and content-based recommendation approaches
"""

import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
import warnings
warnings.filterwarnings('ignore')


class HybridRecommender:
    """Hybrid recommendation system combining multiple recommendation
    strategies."""

    def __init__(self):
        """Initialize the hybrid recommender system."""
        # Data storage
        self.movies_df: pd.DataFrame = None
        self.ratings_df: pd.DataFrame = None

        # Collaborative filtering matrices
        self.user_item_matrix: pd.DataFrame = None
        self.user_similarity_matrix: pd.DataFrame = None
        self.item_similarity_matrix: pd.DataFrame = None

        # Content-based filtering components
        self.tfidf_matrix = None
        self.content_similarity_matrix = None

```

```
        # Dimensionality reduction
        self.svd_model = None

    def load_data(self, movies_path: str = 'ml-latest-small/movies.csv',
                  ratings_path: str = 'ml-latest-small/ratings.csv') -> None:
        """
        Load MovieLens dataset from CSV files.

        Args:
            movies_path: Path to movies CSV file
            ratings_path: Path to ratings CSV file
        """
        print("\n[INFO] Loading dataset...")
        try:
            self.movies_df = pd.read_csv(movies_path)
            self.ratings_df = pd.read_csv(ratings_path)
            print(f"[SUCCESS] Dataset loaded: {len(self.movies_df):,} movies, {len(self.ratings_df):,} ratings")
        except FileNotFoundError as e:
            print(f"[ERROR] File not found: {e}")
            raise
        except Exception as e:
            print(f"[ERROR] Failed to load data: {e}")
            raise

    def preprocess_data(self) -> None:
        """
        Preprocess data and compute similarity matrices for recommendation algorithms.
        This includes user-item matrix creation, similarity computations, and dimensionality reduction.
        """
        print("\n[INFO] Starting data preprocessing...")

        # Create user-item interaction matrix
        # print("[STEP 1/5] Creating user-item matrix...")
        self.user_item_matrix = self.ratings_df.pivot_table(
            index='userId',
```

```
            columns='movieId',
            values='rating',
            fill_value=0
        )

        # Compute user-user similarity matrix (collaborative filtering)
        # print("[STEP 2/5] Computing user similarity matrix...")
        user_similarity_array = cosine_similarity(self.user_item_matrix)
        self.user_similarity_matrix = pd.DataFrame(
            user_similarity_array,
            index=self.user_item_matrix.index,
            columns=self.user_item_matrix.index
        )

        # Compute item-item similarity matrix (collaborative filtering)
        # print("[STEP 3/5] Computing item similarity matrix...")
        item_user_matrix = self.user_item_matrix.T
        item_similarity_array = cosine_similarity(item_user_matrix)
        self.item_similarity_matrix = pd.DataFrame(
            item_similarity_array,
            index=item_user_matrix.index,
            columns=item_user_matrix.index
        )

        # Compute content-based similarity using movie genres
        # print("[STEP 4/5] Computing content-based similarity...")
        self.movies_df['genres'] = self.movies_df['genres'].fillna('')
        tfidf_vectorizer = TfidfVectorizer(stop_words='english')
        self.tfidf_matrix = tfidf_vectorizer.fit_transform(self.movies_df['
    genres'])
        self.content_similarity_matrix = cosine_similarity(self.tfidf_matrix)

        # Apply SVD for dimensionality reduction (memory optimization)
        # print("[STEP 5/5] Applying SVD dimensionality reduction...")
        self.svd_model = TruncatedSVD(n_components=50, random_state=42)
        _ = self.svd_model.fit_transform(self.user_item_matrix)

        print("[SUCCESS] Data preprocessing completed!\n")
```

```
105
106     def user_based_collaborative_filtering(self, user_id: int,
    n_recommendations: int = 10) -> list:
107         """
108         Generate recommendations using user-based collaborative filtering.
109
110         Args:
111             user_id: Target user ID
112             n_recommendations: Number of recommendations to generate
113
114         Returns:
115             List of recommended movie IDs
116         """
117         # Validate user existence
118         if user_id not in self.user_similarity_matrix.index:
119             print(f"[WARNING] User {user_id} not found in similarity matrix")
120             return []
121
122         # Find top similar users (excluding the target user)
123         similar_users = self.user_similarity_matrix.loc[user_id].sort_values(
124             ascending=False
125         ).iloc[1:11]
126
127         # Aggregate recommendations from similar users
128         recommendation_scores = {}
129         for similar_user_id, similarity_score in similar_users.items():
130             # Get movies rated by similar user
131             similar_user_ratings = self.user_item_matrix.loc[similar_user_id]
132             rated_movies = similar_user_ratings[similar_user_ratings > 0].
    index
133
134             # Score unrated movies for target user
135             for movie_id in rated_movies:
136                 if self.user_item_matrix.loc[user_id, movie_id] == 0:  # User
     hasn't rated this movie
137                     if movie_id not in recommendation_scores:
138                         recommendation_scores[movie_id] = 0
139                     recommendation_scores[movie_id] += similarity_score *
```

```
    similar_user_ratings[movie_id]

        # Sort and return top recommendations
        sorted_recommendations = sorted(
            recommendation_scores.items(),
            key=lambda x: x[1],
            reverse=True
        )
        return [movie_id for movie_id, _ in sorted_recommendations[:
    n_recommendations]]

    def item_based_collaborative_filtering(self, user_id: int,
    n_recommendations: int = 10) -> list:
        """
        Generate recommendations using item-based collaborative filtering.

        Args:
            user_id: Target user ID
            n_recommendations: Number of recommendations to generate

        Returns:
            List of recommended movie IDs
        """
        # Validate user existence
        if user_id not in self.user_item_matrix.index:
            print(f"[WARNING] User {user_id} not found in user-item matrix")
            return []

        # Get user's rated movies
        user_ratings = self.user_item_matrix.loc[user_id]
        rated_movies = user_ratings[user_ratings > 0].index

        # Aggregate recommendations based on similar items
        recommendation_scores = {}
        for rated_movie_id in rated_movies:
            if rated_movie_id not in self.item_similarity_matrix.index:
                continue

```

```
            # Find similar movies to each rated movie
            similar_movies = self.item_similarity_matrix.loc[rated_movie_id].sort_values(
                ascending=False
            ).iloc[1:11]

            # Score unrated similar movies
            for movie_id, similarity_score in similar_movies.items():
                if user_ratings.get(movie_id, 0) == 0:  # User hasn't rated this movie
                    if movie_id not in recommendation_scores:
                        recommendation_scores[movie_id] = 0
                    recommendation_scores[movie_id] += similarity_score * user_ratings[rated_movie_id]

        # Sort and return top recommendations
        sorted_recommendations = sorted(
            recommendation_scores.items(),
            key=lambda x: x[1],
            reverse=True
        )
        return [movie_id for movie_id, _ in sorted_recommendations[:n_recommendations]]

    def content_based_recommendation(self, movie_id: int, n_recommendations: int = 10) -> list:
        """
        Generate recommendations based on content similarity (movie genres).

        Args:
            movie_id: Reference movie ID
            n_recommendations: Number of recommendations to generate

        Returns:
            List of similar movie IDs
        """
        # Validate movie existence
        if movie_id not in self.movies_df['movieId'].values:
```

```
208             print(f"[WARNING] Movie {movie_id} not found in dataset")
209             return []
210 
211         # Find movie index and get similarity scores
212         movie_idx = self.movies_df[self.movies_df['movieId'] == movie_id].
    index[0]
213         similarity_scores = list(enumerate(self.content_similarity_matrix[
    movie_idx]))
214 
215         # Sort by similarity and exclude the reference movie
216         similarity_scores = sorted(similarity_scores, key=lambda x: x[1],
    reverse=True)[1:n_recommendations+1]
217 
218         # Return similar movie IDs
219         return [self.movies_df.iloc[idx]['movieId'] for idx, _ in
    similarity_scores]
220 
221     def hybrid_recommendation(self, user_id: int, n_recommendations: int =
    10,
222                               cf_weight: float = 0.6, content_weight: float =
    0.4) -> list:
223         """
224         Generate hybrid recommendations combining multiple approaches.
225 
226         Args:
227             user_id: Target user ID
228             n_recommendations: Number of recommendations to generate
229             cf_weight: Weight for collaborative filtering component
230             content_weight: Weight for content-based component
231 
232         Returns:
233             List of recommended movie IDs
234         """
235         print(f"\n[INFO] Generating hybrid recommendations for user {user_id
    }...")
236 
237         # Get user's rating history
238         user_history = self.ratings_df[self.ratings_df['userId'] == user_id]
```

```
239
240         # Handle cold-start scenario (users with limited history)
241         if len(user_history) < 5:
242             cf_weight = 0.4
243             content_weight = 0.3
244             popularity_weight = 0.3
245             print("[NOTICE] Cold-start user detected. Adjusting
    recommendation weights...")
246         else:
247             popularity_weight = 0.0
248
249         # Step 1: Collaborative filtering recommendations
250         print("[HYBRID] Computing collaborative filtering recommendations..."
    )
251         cf_recommendations = {}
252
253         # User-based CF
254         user_cf_recs = self.user_based_collaborative_filtering(user_id,
    n_recommendations * 2)
255         for movie_id in user_cf_recs:
256             cf_recommendations[movie_id] = cf_recommendations.get(movie_id,
    0) + 1.0
257
258         # Item-based CF
259         item_cf_recs = self.item_based_collaborative_filtering(user_id,
    n_recommendations * 2)
260         for movie_id in item_cf_recs:
261             cf_recommendations[movie_id] = cf_recommendations.get(movie_id,
    0) + 1.0
262
263         # Step 2: Content-based recommendations
264         print("[HYBRID] Computing content-based recommendations...")
265         content_recommendations = {}
266         if len(user_history) > 0:
267             # Use top 3 recently watched movies for content-based
    recommendations
268             recent_movies = user_history.nlargest(3, 'timestamp')['movieId'].
    values
```

```
            for movie_id in recent_movies:
                content_recs = self.content_based_recommendation(movie_id,
    n_recommendations)
                for rec_movie_id in content_recs:
                    content_recommendations[rec_movie_id] =
    content_recommendations.get(rec_movie_id, 0) + 1.0

        # Step 3: Popularity-based recommendations (for cold-start)
        print("[HYBRID] Computing popularity-based recommendations...")
        popularity_recommendations = {}
        if popularity_weight > 0:
            movie_popularity = self.ratings_df.groupby('movieId').size()
            popular_movies = movie_popularity.nlargest(n_recommendations * 2)
    .index
            for movie_id in popular_movies:
                popularity_recommendations[movie_id] = 1.0

        # Step 4: Combine scores with weighted fusion
        print("[HYBRID] Combining recommendation scores...")
        combined_scores = {}

        # Add collaborative filtering scores
        for movie_id, score in cf_recommendations.items():
            combined_scores[movie_id] = combined_scores.get(movie_id, 0) +
    score * cf_weight

        # Add content-based scores
        for movie_id, score in content_recommendations.items():
            combined_scores[movie_id] = combined_scores.get(movie_id, 0) +
    score * content_weight

        # Add popularity scores
        for movie_id, score in popularity_recommendations.items():
            combined_scores[movie_id] = combined_scores.get(movie_id, 0) +
    score * popularity_weight

        # Step 5: Apply diversity penalty to avoid genre over-concentration
        print("[HYBRID] Applying diversity optimization...")
```

```
        final_recommendations = []
        genre_frequency = {}

        for movie_id, score in sorted(combined_scores.items(), key=lambda x: x[1], reverse=True):
            if len(final_recommendations) >= n_recommendations:
                break

            # Get movie information
            movie_info = self.movies_df[self.movies_df['movieId'] == movie_id]
            if len(movie_info) == 0:
                continue

            # Extract genres
            genres = movie_info.iloc[0]['genres'].split('|')

            # Calculate diversity penalty based on genre frequency
            diversity_penalty = 0
            for genre in genres:
                if genre in genre_frequency:
                    diversity_penalty += genre_frequency[genre] * 0.1

            # Apply penalty and filter
            final_score = score - diversity_penalty
            if final_score > 0:
                final_recommendations.append(movie_id)
                for genre in genres:
                    genre_frequency[genre] = genre_frequency.get(genre, 0) + 1

        print(f"[SUCCESS] Generated {len(final_recommendations)} hybrid recommendations")
        return final_recommendations

    def get_movie_details(self, movie_ids: list) -> list:
        """
        Retrieve detailed information for a list of movie IDs.
```

```
335
336         Args:
337             movie_ids: List of movie IDs
338
339         Returns:
340             List of dictionaries containing movie details
341         """
342         movie_details = []
343         for movie_id in movie_ids:
344             movie_data = self.movies_df[self.movies_df['movieId'] == movie_id
    ]
345             if len(movie_data) > 0:
346                 movie_row = movie_data.iloc[0]
347                 movie_details.append({
348                     'movie_id': int(movie_row['movieId']),
349                     'title': str(movie_row['title']),
350                     'genres': str(movie_row['genres'])
351                 })
352         return movie_details
353
354     def evaluate_recommendations(self, user_id: int, test_ratio: float = 0.2)
     -> dict:
355         """
356         Evaluate recommendation quality using hold-out validation.
357
358         Args:
359             user_id: Target user ID
360             test_ratio: Proportion of recent ratings to use as test set
361
362         Returns:
363             Dictionary containing evaluation metrics
364         """
365         # Get user's rating history
366         user_ratings = self.ratings_df[self.ratings_df['userId'] == user_id]
367
368         # Validate sufficient data
369         if len(user_ratings) < 10:
370             return {"error": "Insufficient user rating data for evaluation"}
```

```
371
372         # Split into training and test sets
373         test_size = int(len(user_ratings) * test_ratio)
374         if test_size <= 0:
375             return {"error": "Test set size is zero"}
376
377         # Use most recent ratings as test set
378         test_movies = user_ratings.nlargest(test_size, 'timestamp')['movieId'
    ].values
379
380         # Temporarily modify dataset for evaluation
381         original_ratings = self.ratings_df.copy()
382
383         try:
384             # Create training set by removing test ratings
385             train_ratings = self.ratings_df[~(
386                 (self.ratings_df['userId'] == user_id) &
387                 (self.ratings_df['movieId'].isin(test_movies))
388             )]
389
390             # Rebuild models with training data only
391             self.ratings_df = train_ratings
392             self.preprocess_data()
393
394             # Generate recommendations
395             recommendations = self.hybrid_recommendation(user_id,
    n_recommendations=20)
396
397         finally:
398             # Restore original dataset
399             self.ratings_df = original_ratings
400             self.preprocess_data()
401
402         # Calculate hit rate
403         hits = len(set(recommendations) & set(test_movies))
404         hit_rate = hits / len(test_movies) if len(test_movies) > 0 else 0
405
406         return {
```

```
            "user_id": user_id,
            "test_set_size": len(test_movies),
            "recommendation_count": len(recommendations),
            "hits": hits,
            "hit_rate": hit_rate,
            "precision": hits / len(recommendations) if len(recommendations)
    > 0 else 0
        }

    def evaluate_single_algorithm(self, user_id: int, algorithm: str,
                                  test_ratio: float = 0.2, n_recommendations:
     int = 10) -> dict:
        """
        Evaluate performance of a single recommendation algorithm.

        Args:
            user_id: Target user ID
            algorithm: Algorithm name ('user_cf', 'item_cf', or 'content')
            test_ratio: Proportion of recent ratings for testing
            n_recommendations: Number of recommendations to generate

        Returns:
            Dictionary containing evaluation metrics
        """
        # Get user's rating history
        user_ratings = self.ratings_df[self.ratings_df['userId'] == user_id]

        # Validate sufficient data
        if len(user_ratings) < 10:
            return None

        # Prepare test set
        test_size = int(len(user_ratings) * test_ratio)
        if test_size <= 0:
            return None

        test_movies = user_ratings.nlargest(test_size, 'timestamp')['movieId'
    ].values
```

```
442
443         # Temporarily modify dataset
444         original_ratings = self.ratings_df.copy()
445
446         try:
447             # Create training set
448             train_ratings = self.ratings_df[~(
449                 (self.ratings_df['userId'] == user_id) &
450                 (self.ratings_df['movieId'].isin(test_movies))
451             )]
452             self.ratings_df = train_ratings
453             self.preprocess_data()
454
455             # Generate recommendations based on algorithm
456             recommendations = []
457             if algorithm == 'user_cf':
458                 recommendations = self.user_based_collaborative_filtering(
    user_id, n_recommendations)
459             elif algorithm == 'item_cf':
460                 recommendations = self.item_based_collaborative_filtering(
    user_id, n_recommendations)
461             elif algorithm == 'content':
462                 recent_movies = user_ratings.nlargest(3, 'timestamp')['
    movieId'].values
463                 if len(recent_movies) > 0:
464                     recommendations = self.content_based_recommendation(
    recent_movies[0], n_recommendations)
465
466         finally:
467             # Restore original dataset
468             self.ratings_df = original_ratings
469             self.preprocess_data()
470
471         # Calculate recall
472         hits = len(set(recommendations) & set(test_movies))
473         recall = hits / len(test_movies) if len(test_movies) > 0 else 0
474
475         return {
```

```
            "algorithm": algorithm,
            "recall": recall,
            "hits": hits,
            "test_set_size": len(test_movies),
            "recommendation_count": len(recommendations)
        }


def display_recommendations(recommender: HybridRecommender, user_id: int,
                            recommendations: list) -> None:
    """
    Display formatted recommendation results for a user.

    Args:
        recommender: HybridRecommender instance
        user_id: User ID
        recommendations: List of recommended movie IDs
    """
    if not recommendations:
        print(f"  [INFO] No recommendations generated for user {user_id}")
        return

    print(f"\n  Recommended Movies for User {user_id}:")
    print("  " + "-" * 60)

    movie_details = recommender.get_movie_details(recommendations)
    for idx, movie in enumerate(movie_details, 1):
        print(f"  {idx:2d}. {movie['title'][:50]:<50} | {movie['genres']}")

    print("  " + "-" * 60)


def display_user_history(recommender: HybridRecommender, user_id: int) ->
    None:
    """
    Display user's recent viewing history.

    Args:
```

```
        recommender: HybridRecommender instance
        user_id: User ID
    """
    user_history = recommender.ratings_df[recommender.ratings_df['userId'] ==
     user_id]

    if len(user_history) == 0:
        print(f"  [INFO] No viewing history found for user {user_id}")
        return

    print(f"\n  User {user_id} Viewing History:")
    print(f"  Total ratings: {len(user_history):,}")

    if len(user_history) > 0:
        print(f"\n  Recently Watched:")
        recent_movies = user_history.nlargest(3, 'timestamp')
        for _, rating_row in recent_movies.iterrows():
            movie_info = recommender.movies_df[
                recommender.movies_df['movieId'] == rating_row['movieId']
            ]
            if len(movie_info) > 0:
                title = movie_info.iloc[0]['title']
                print(f"    • {title[:45]:<45} (Rating: {rating_row['rating
    ']:.1f})")


def display_evaluation_results(evaluation: dict, algorithm_name: str = "
    Hybrid") -> None:
    """
    Display formatted evaluation results.

    Args:
        evaluation: Evaluation metrics dictionary
        algorithm_name: Name of the evaluated algorithm
    """
    print(f"\n  {algorithm_name} Algorithm Evaluation:")
    print("  " + "-" * 40)
    print(f"    Test Set Size:     {evaluation.get('test_set_size', 0):>6}")
```

```
548     print(f"    Hits:              {evaluation.get('hits', 0):>6}")
549     print(f"    Hit Rate:          {evaluation.get('hit_rate', 0):>6.2%}")
550     if 'recall' in evaluation:
551         print(f"    Recall:            {evaluation['recall']:>6.2%}")
552     if 'precision' in evaluation:
553         print(f"    Precision:         {evaluation['precision']:>6.2%}")
554
555
556 def main():
557     """Main execution function for the hybrid recommendation system."""
558
559     print("\n" + "=" * 70)
560     print(" " * 15 + "HYBRID MOVIE RECOMMENDATION SYSTEM")
561     print("=" * 70)
562
563     # Initialize recommender system
564     print("\n[SYSTEM] Initializing recommendation engine...")
565     recommender = HybridRecommender()
566
567     # Load and preprocess data
568     try:
569         recommender.load_data()
570         recommender.preprocess_data()
571     except Exception as e:
572         print(f"[ERROR] System initialization failed: {e}")
573         return
574
575     # Demonstration with sample users
576     sample_users = [4, 16, 27, 69, 99, 101, 110, 120, 134, 180]
577     print(f"[SYSTEM] Starting recommendation demonstration for {len(
        sample_users)} users...\n")
578
579     for user_id in sample_users:
580         print(f"\n{'=' * 70}")
581         print(f"USER {user_id:4d} RECOMMENDATION SESSION")
582         print(f"{'=' * 70}")
583
584         # Display user history
```

```
585         display_user_history(recommender, user_id)
586 
587         # Generate and display hybrid recommendations
588         print(f"\n[RECOMMENDER] Generating hybrid recommendations...")
589         recommendations = recommender.hybrid_recommendation(user_id,
    n_recommendations=10)
590         display_recommendations(recommender, user_id, recommendations)
591 
592         # Evaluate recommendation quality for users with sufficient data
593         user_history = recommender.ratings_df[recommender.ratings_df['userId'
    ] == user_id]
594         if len(user_history) >= 10:
595             print(f"\n[RECOMMENDER] Evaluating recommendation quality...")
596 
597             # Evaluate hybrid approach
598             hybrid_evaluation = recommender.evaluate_recommendations(user_id)
599             if 'error' not in hybrid_evaluation:
600                 display_evaluation_results(hybrid_evaluation, "Hybrid")
601 
602             # Compare individual algorithms
603             print(f"\n  Individual Algorithm Comparison:")
604             print("  " + "-" * 40)
605 
606             algorithms = ['user_cf', 'item_cf', 'content']
607             algorithm_names = {
608                 'user_cf': 'User-Based CF',
609                 'item_cf': 'Item-Based CF',
610                 'content': 'Content-Based'
611             }
612 
613             for algo in algorithms:
614                 eval_result = recommender.evaluate_single_algorithm(user_id,
    algo)
615                 if eval_result:
616                     recall_pct = eval_result['recall'] * 100
617                     print(f"    {algorithm_names[algo]:<18} Recall: {
    recall_pct:>5.1f}%")
618 
```

```
619     print("\n" + "=" * 70)
620     print(" " * 20 + "RECOMMENDATION DEMONSTRATION COMPLETED")
621     print("=" * 70)
622     print("\n[SYSTEM] Thank you for using the Hybrid Movie Recommendation
        System!\n")
623
624
625 if __name__ == "__main__":
626     main()
```